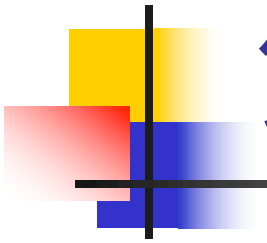


言語処理系 第4回
LR構文解析(1)

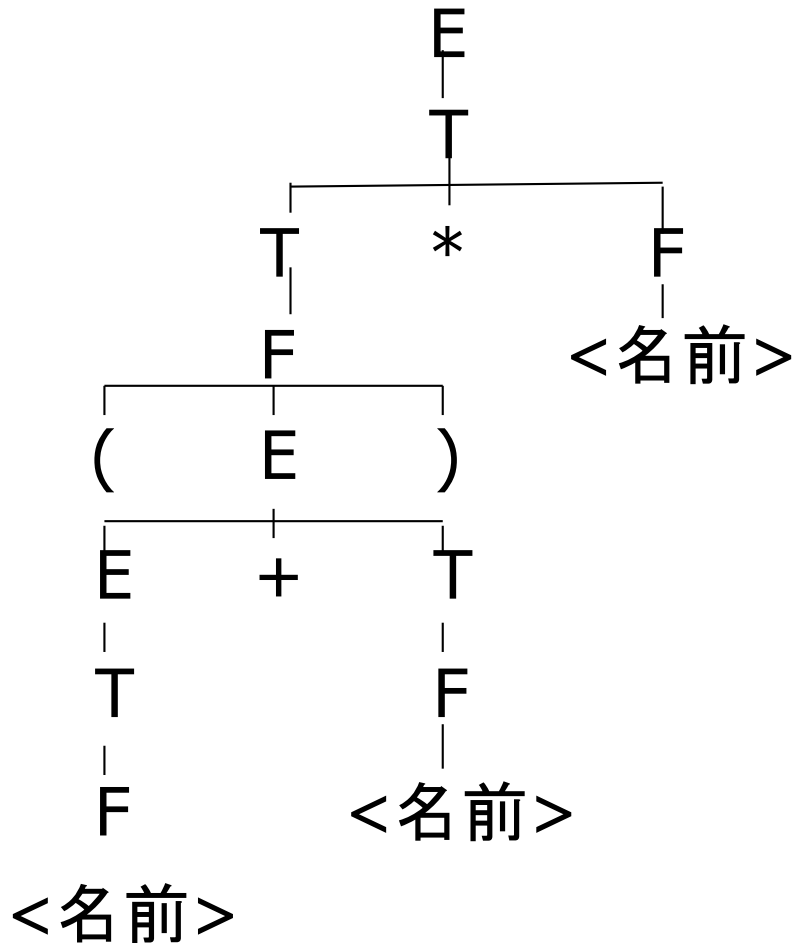
木村啓二
keiji@waseda.jp

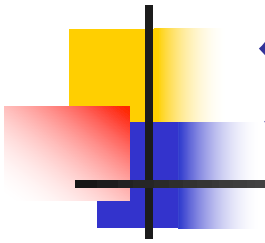


第3回課題(1)解答

- (i+j)*kに先に示した文法Gの生成規則を適用して解析木を構築する。Gの開始記号はEである。これに関して以下の問いに答えよ。
- 1. 最初に適用できる生成規則は何か
  - E→T
- 2. 2番目に適用できる生成規則は何か
  - T→T*F
- 3. (i+j)の導出ではF→(E)を適用するが、2番目の生成規則からF→(E)を適用するまでに必要となる生成規則を全て答えよ
  - T→F

第3回課題(1)解析木（参考）



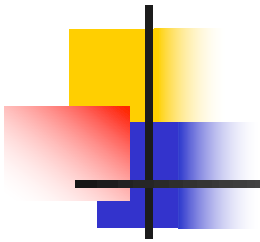


第3回課題(2)

- 資料20ページの文法G2のFIRST, FOLLOW, DIRECTORをそれぞれ示せ（＜名前＞＜数字＞はiで表す）
  - Moodle上では一部だけ答えるように設定するが、各自で実際に一度は全て求めることを推奨する

第3回課題(2)解答例

- G2
  - First(F)={(,i}, First(T')={*,ε}, First(T)={(,i}, First(E')={+,ε}, First(E)={(,i}
  - Follow(E)={$,)}, Follow(E')={$,)}, Follow(T)={+,$,)}, Follow(T')={+,$,)}, Follow(F)={*,+,$,)}
  - Director(E,TE')={(,i},
Director(E',+TE')={+}, Director(E',ε)={$,)},
Director(T,FT')={(,i},
Director(T',*FT')={*}, Director(T',ε)={+,$,)},
Director(F,(E))={(}, Director(F,i)={i}
- （参考）G2の生成規則
  - E→TE', E'→+TE'|ε
T→FT', T'→*FT'|ε
F→(E)|i（<数字>もしくは<名前>）

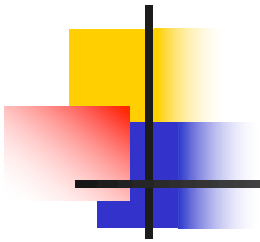


字句解析・構文解析

- コンパイラのフロントエンド部の処理
- ソースプログラムはファイル上の単なる文字列
  - 文字列から「書いてあること」を解析する必要がある
- 字句解析
  - ソースプログラムから単語を切り出す
- 構文解析
  - 式の解析が大きな動機の一つ
  - 字句解析で切り出した単語のつながりから、構文規則(syntax)をチェックする
    - 字句間のつながり方（式の場合、演算順序も表現する）をチェックする
  - さらにプログラムを内部表現に落とし込む

式の計算(評価)順序の解釈は？（再掲）

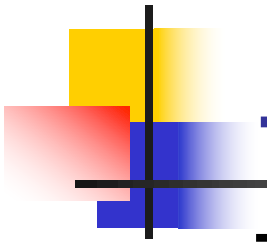
- 以下の式が入力として与えられたとする
  - a+b*c
- 人間はb*cを先に計算することがすぐにわかる
- 機械はa, +, b, *, cと順番に読むしかない
  - 実際には字句解析結果の記号の順番として読み出す
    - 名前, +, 名前, *, 名前
- a, +まで読み取って、次にbを読み取ったときに、a+bを計算して良いか？
  - 当然駄目。ではどうする？
  - 1) bの後に*を読み取るかもしれないという前提で記号を読む
  - **2) bの後に読み取る可能性のある記号列をいくつか考慮して記号を読む→本日のアプローチ**



LR(k)文法

- 左から右へトークンをk個先読みしつつ最右導出できる文法のクラス
  - LL(k)文法より表現できる範囲が広い
- 上向き構文解析で解析器を実現
- 今回は以下を取り扱う
  - SLR(1) ←本日はここまで
  - LR(1)
  - LALR(1)（オプション）
- 解析器の自動生成ツールがある

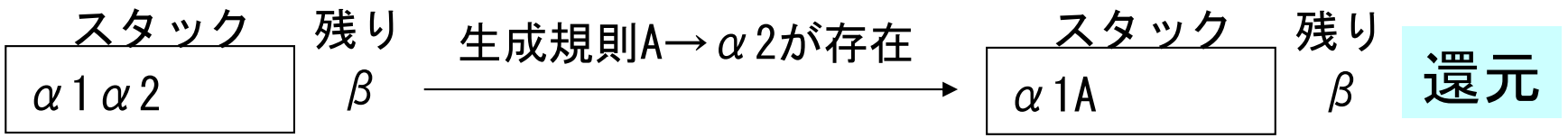
上向き構文解析の基本戦略



- プログラムを左から右に走査しながら、還元できるものを順次還元する
- 還元できなければ，**そのうち還元できるかもしれない**生成規則群を記録してプログラムを読み進める
  - **生成規則群を一つの状態とする状態遷移図を構成する**
  - 生成規則適用を保留して読み進める
- 記録場所にスタックを利用する
  - 読み進めた過程の情報もスタックに記録する

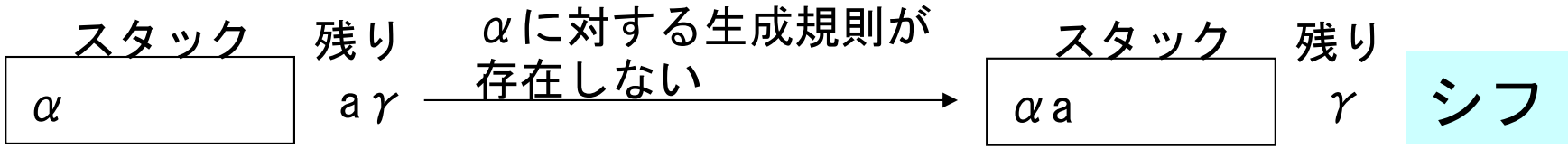
上向き構文解析を行う プログラムの動作イメージ

$\alpha 1 \alpha 2 \beta$ とプログラムがあり、$\alpha 1 \alpha 2$まで読んだ状態



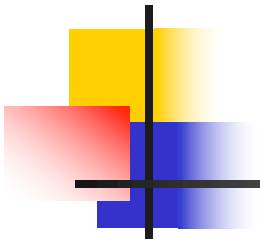
還元

$\alpha a \gamma$ とプログラムがあり、αまで読んだ状態



シフト

還元・シフトを繰り返しながら解析する（シフト還元構文解析）



LR構文解析の概略

- LR文法
  - 各時点でのシフト・還元いずれかが可能
  - Left-to-right scanning, Rightmost derivation in reverse
- 生成規則よりシフト及び還元の状態遷移表を作成する
  - T→T*Fにおいて、*読み込みにより遷移できる状態は {T→T*.F,F→.(E),F→.i} (iは<名前>もしくは<数字>)
  - ただし「.」は次の読み込み位置
  - 語彙の読み出しにより遷移できる状態の集合を求める
- 最右導出
  - スタックを使うので
- シフト・還元の状態遷移の衝突回避の方法によりいくつかのアルゴリズムがある
  - SLR, LALR

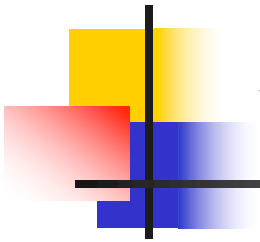
状態遷移表の作成準備 LR(0)項とclosure(I)

- LR(0)項
  - 文法G={P,S}におけるPの任意の生成規則に対して、その右辺の左端、右端、および終端記号または非終端記号の間のどれか1カ所に「.」を付けたもの（読み出し位置）
  - 例：A→xyzのLR(0)項
    A→.xyz, A→x.yz, A→xy.z, A→xyz.#
    ← 還元すべき状態（還元状態）を表す
- closure(I)
  （LR(0)項の集合Iの閉包: Iから到達できるLR(0)項の集合）
  →「.」の右側にどのような記号が出てくるか追いかけていく
  - 1. closure(I) = I
  - 2. A→α.Bβ ∈closure(I)に対してB→γという生成規則があればB→.γをclosure(I)に加える
  - 新しく追加するものがなくなるまで2を繰り返す
  - 例：G1={P,E}, P={E→E+T|T, T→T*F|F, F→(E)|i}において
    I={E→E+.T}の閉包
    closure({E→E+.T})={E→E+.T, T→.T*F, T→.F,F→.(E),F→.i}
  - **closure(I)が状態遷移表の状態に対応**

状態遷移表の作成

- GOTO(I,X)の定義
  - ただしIはLR(0)項の集合、$X \in (V_N \cup V_T) = V$
  - GOTO(I,X)
    =closure({$A \rightarrow \alpha X.\ \beta \mid A \rightarrow \alpha .\ X\beta \in I$})
  - 状態I(のLR0項)からXの読み込みによって遷移するLR0項の集合
    - 状態I(LR(0)項の集合)からのXによる新たな状態(これもLR(0)項の集合
  - dragon bookの記述がわかりやすい
    - GOTO(I,X) is defined to be the closure of the set of all items [$A \rightarrow \alpha X.\ \beta$] such that [$A \rightarrow \alpha .\ X\beta$] is in I.
  - 文法Gにおける全てのGOTO(I,X)を求めれば状態遷移表は完成

状態を求めるアルゴリズム



- 文法G={P,S}を対象
  - S′→S$をGに追加
    G′={P∪{S′→S$}, S′}
  - 初期設定: C={clousure({S′→.S$})}
  - 以下を新たな集合がCに付け加えられなくなるまで繰り返す
    - 任意のI∈CとX∈Vに対し、GOTO(I,X)が空集合でなく、Cに入っていなかったらそれをCに加える

サンプルの文法

- G={P,E}
- P={E→E+T
  E→T
  T→T*F
  T→F
  F→(E)
  F→i

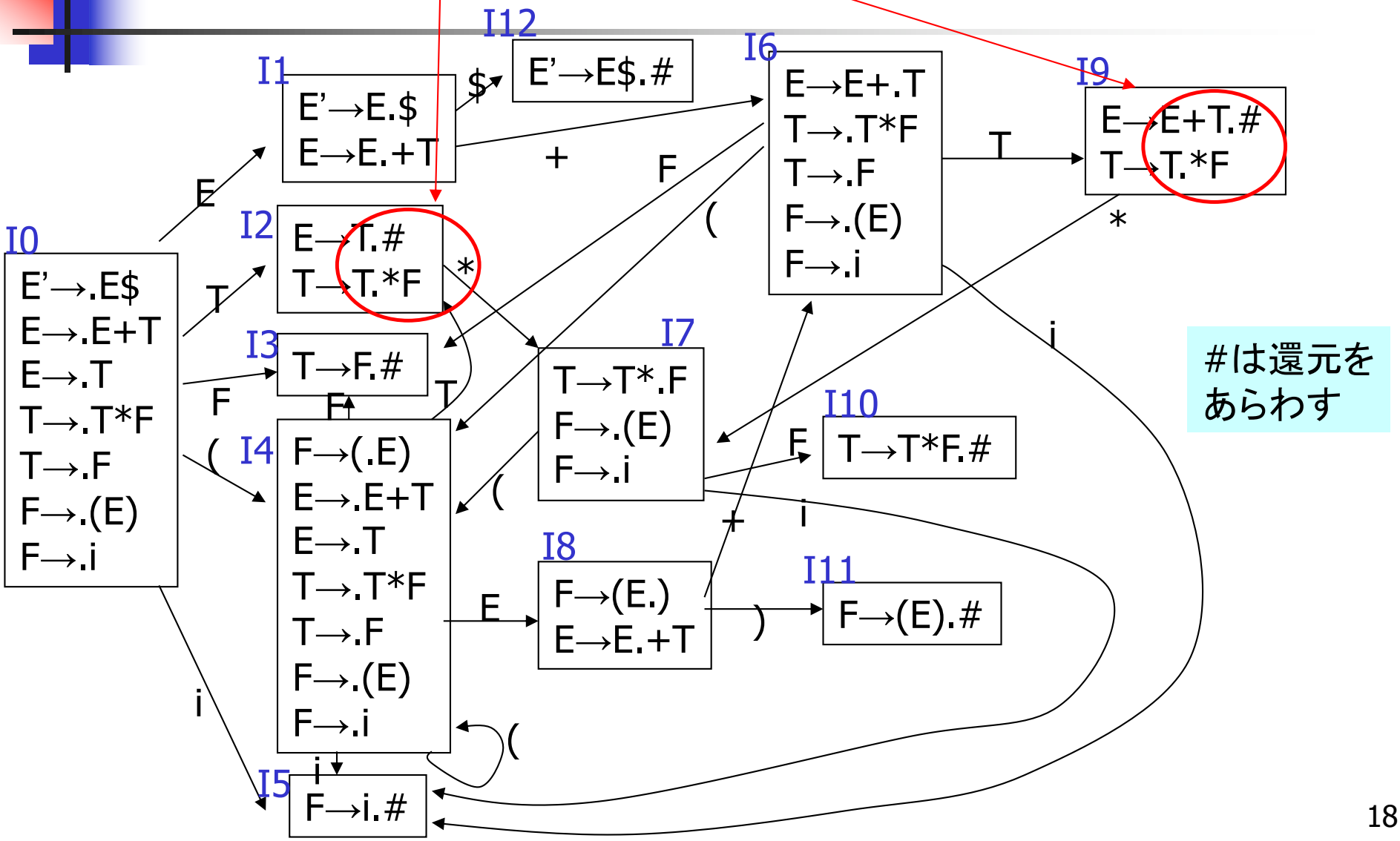
- iは<名前>もしくは<数字>
- +,*,(,),iは終端記号

例

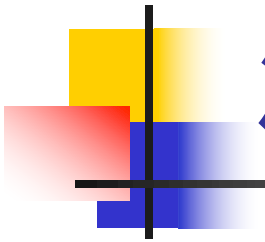
- C={closure({E′→.E$})}（初期状態）
- closure({E′→.E$})={E′→.E$, E→.E+T, E→.T, T→.T*F, T→.F, F→.(E), F→.i}
  - これをI0とする
- GOTO(I0, E) = GOTO({E′→.E$, E→.E+T}, E) = closure({E′→E.$, E→E.+T}) = {E′→E.$, E→E.+T}
  - $はこれで終了、+の読み出しに対応する生成規則は無し
  - これをI1とする

とりあえず先読みして
シフト・還元を決定する（ものとする）

文法Gの状態遷移図



#は還元をあらわす

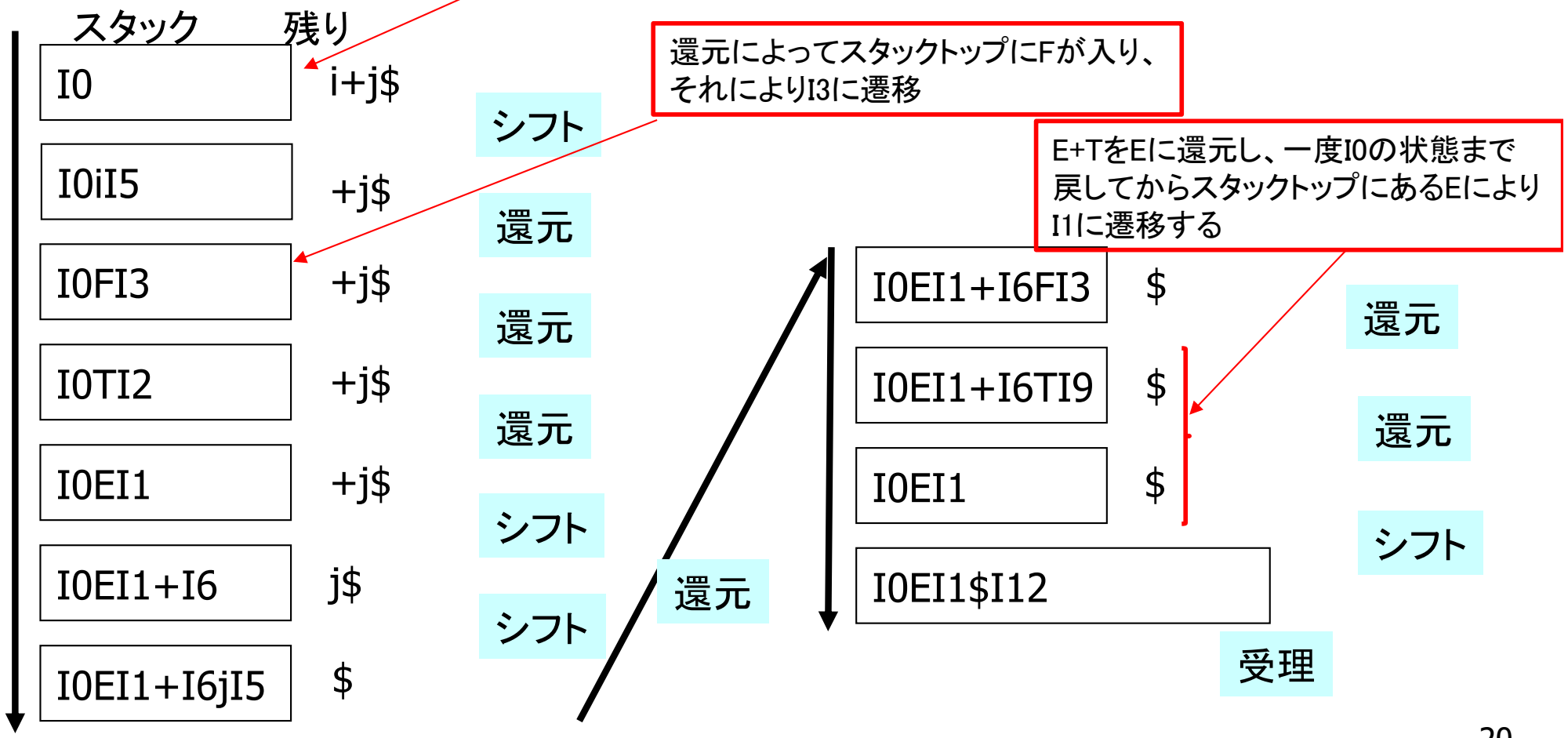


演習問題(1)

- 1. 文法Gのclosure({F→(.E)})を求めよ
- 2. GOTO(I4, ()を求めよ
- 3. GOTO(I4,i)を求めよ
  - Moodleでの回答は少し違う形式で行います
    （もちろん本質的に問うていることは同じ）

シフト・還元の例(1)

- i+j$

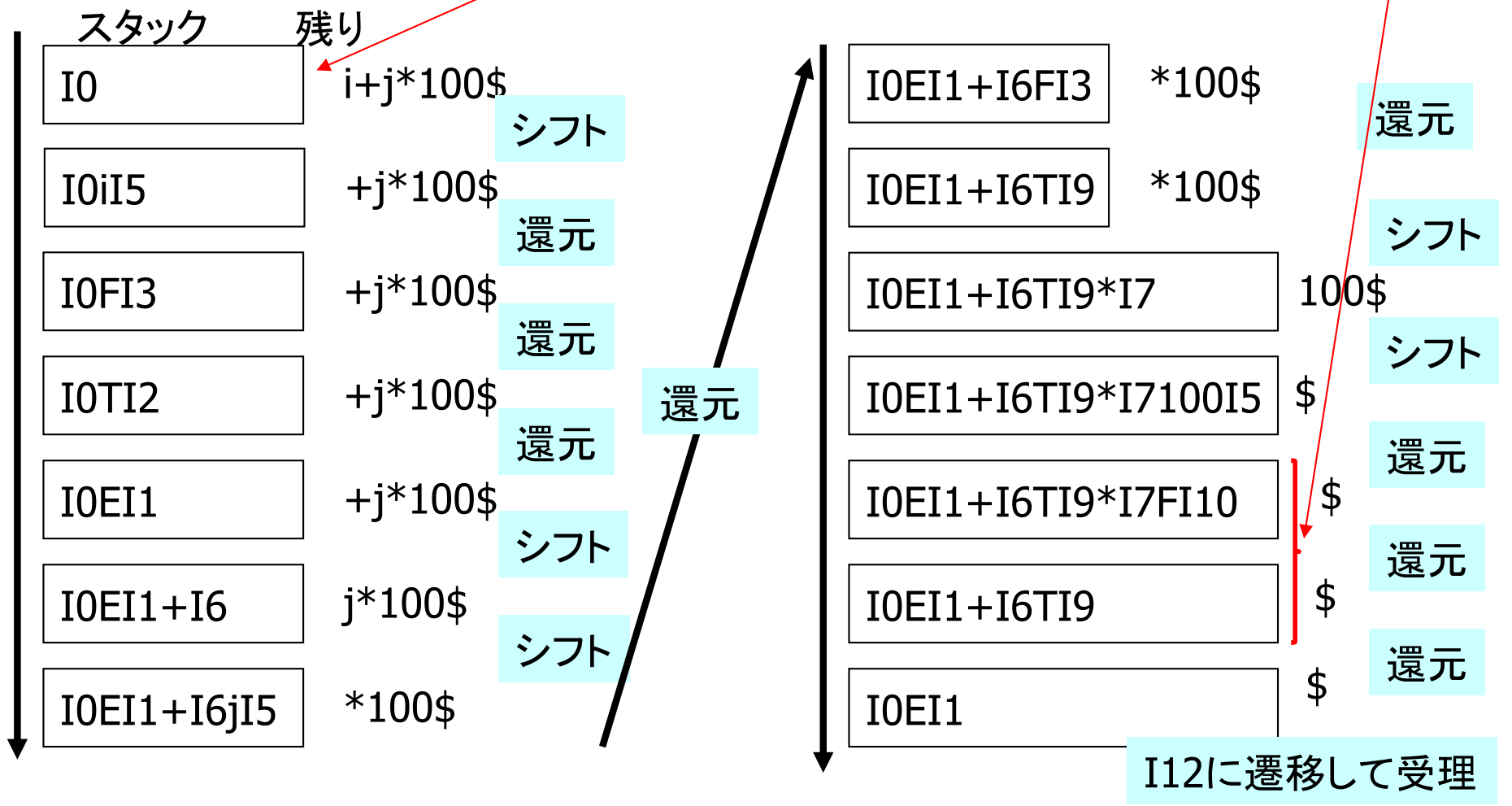

記号と共に状態もスタックに記録していく
（還元したときにどの状態に戻るかわかる）
還元によってスタックトップにFが入り、
それによりI3に遷移
E+TをEに還元し、一度I0の状態まで
戻してからスタックトップにあるEにより
I1に遷移する
スタック
残り
I0
i+j$
シフト
I0iI5
+j$
還元
I0FI3
+j$
還元
I0TI2
+j$
還元
I0EI1
+j$
シフト
I0EI1+I6
j$
シフト
I0EI1+I6jI5
$
還元
I0EI1+I6FI3
$
還元
I0EI1+I6TI9
$
還元
I0EI1
$
シフト
I0EI1$I12
受理

シフト・還元の例(2)

- i+j*100$

記号と共に状態もスタックに記録していく
（還元したときにどの状態に戻るかわかる）

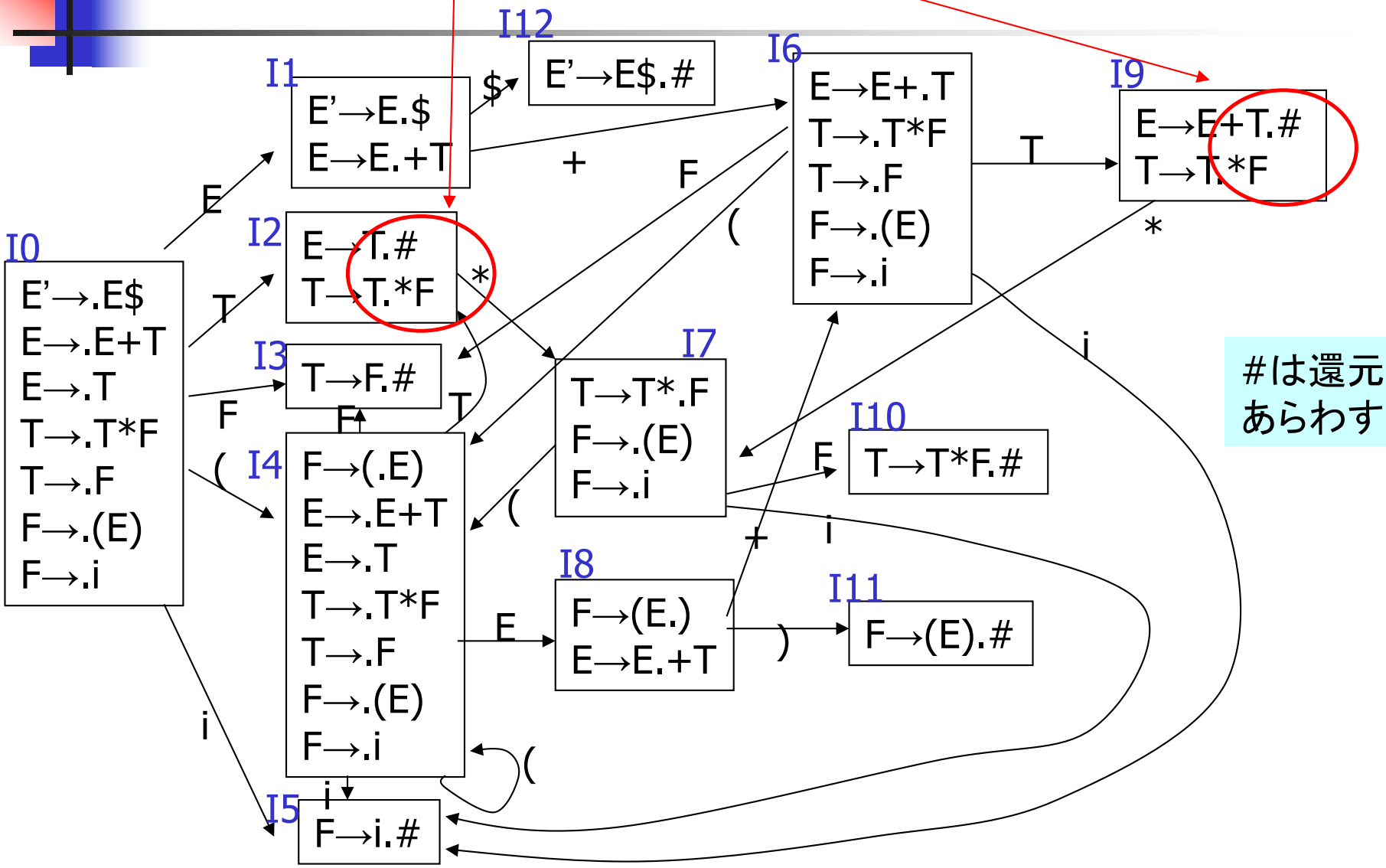
T*FをTに還元し、一度I6の状態まで戻してからスタックトップにあるTによりI9に遷移する



とりあえず先読みして
シフト・還元を決定する（ものとする）

シフト・還元の曖昧性が生じてしまい，問題となる

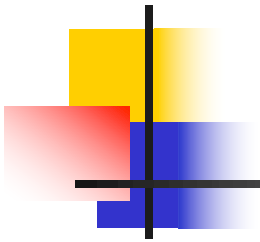
文法Gの状態遷移図



#は還元をあらわす

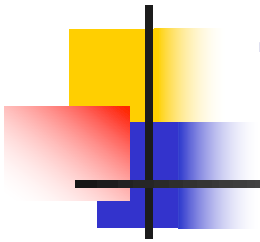
SLR(1) (Simple LR(1))構文解析

- 記号を一つ先読みして構文解析器の動作を決める
- 競合状態
  - シフト/還元競合、還元/還元競合
- 次の1記号を**Follow集合だけ**でLR(0)項による状態遷移図の競合状態を解決できる文法がSLR(1)文法
  - A→α.のときにFollow(A)の終端記号が来たらAに還元する
- 以下の手順で矛盾無くSLR(1)構文解析表が作れることが条件
  - LR(0)状態の集合をC={I0, I1, …, In}， A[Ii, a]はスタックの先頭の状態がIiで次の入力記号がaのときの取るべき動作（このAはActionのA）、sjはシフトして状態Ijに遷移
  - 0. 初期状態はS’ →. S$を含む状態
  - 1. GOTO(Ii, a)=IjならばA[Ii, a]=sj
  - 2. A→α. #∈IiでaεFollow(A)ならばA[Ii, a]={A→αで還元}
  - 3. S’ →S. $∈IiならA[Ii, $]={受理}
- 上記手順でA[Ii,x]（アクションテーブル）を構築する
  - A[Ii, a]が二重に定義されるなど矛盾が生じたらNG



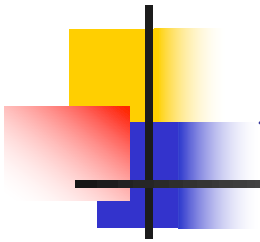
文法Gの状態遷移図に関して

- Follow(E)={+,),$}
- A[I2,+] = A[I2,)] = A[I2, $] = {E→Tで還元}
- A[I2,*]=s7（すなわち，状態7にシフトする）



まとめ

- LR構文解析
  - シフト・還元による状態遷移
  - SLR(1)
- 状態遷移に関してよく理解しておいてください



演習問題(2)

- 文法Gに関してSLR(1)構文解析表を求める
  - A[I9, +], A[I9,)], A[I9, $], A[I9, *]をそれぞれ求めよ
    - ヒント: どの記号のFollow集合を使うか？